

C 语言入门教程

Grant Lee

2025 年 12 月 8 日

目录

1 C 语言程序的基本结构	4
1.1 最小示例 HelloWorld	4
1.2 #include 与头文件	4
1.3 main 函数（程序入口）	4
1.4 语句块与分号	5
1.5 printf 与常用占位符	5
1.6 从源代码到可执行文件：将自然语言翻译为机器语言	5
2 C 语言的数据基石——变量、常量和字面量	8
2.1 数据的原形：字面量 (literal) 的概念与初步认识	8
2.2 字面量的进制与类型区分	10
2.3 浮点数的后缀	16
2.4 前后缀小结	16
2.5 字面量底层机制揭秘（ARM）	17
2.6 特殊字面量：字符串字面量 (String Literals) 底层机制解密	19
2.7 核心疑问解答	20
2.8 字面量底层硬件原理总结：三种字面量的“物流通道”	24
2.9 4. 关键思考：指针 vs 数组	25
2.10 字符串字面量：只读段与栈拷贝 (String Literals)	25
2.11 C99 复合字面量 (Compound Literals)	26
2.12 总结：字面量存储开销对比表	26
2.13 3. 例外情况：字符串与大型常量表	26
2.14 4. 立即数大小受体系结构限制	26
2.15 5. 字面量属于编译期常量	27
2.16 6. 更严谨的总结合述	27
2.17 变量与常量的基本定义	27
3 C 语言的变量与常量	28
3.1 变量 (Variables)	28
3.2 常量 (Constants)	29
3.3 三种定义“常量”符号的方式	29
3.4 总结与对比	31
3.5 命名规则与规范	35
3.6 作用域与生命周期	35

目录	3
4 从字面量 (Literal) 到数据类型	18
4.1 何为魔法数字?	18
4.2 定义与概念	19
4.3 字面量的具体区分方式	19
4.4 浮点数的后缀	23
4.5 总结	23
5 为什么我们需要数据类型	24
5.1 计算机与数学	24
5.2 数据类型的本质: 对比特模式的解释	25
5.3 数据类型决定存储格式与运算规则	26
5.4 有限位宽带来的问题: 溢出、截断与精度损失	27
5.5 小结	31
5.6 整数的存储与表示	32
6 数据类型——基础知识入门	35
6.1 小白需掌握的一些入门知识	35
7 数据类型——整数类型 (Signed Integers)	37
7.1 有符号整数 (Signed Integers)	37
7.2 无符号整数 (Unsigned Integers)	38
8 数据类型——浮点类型 (Floating-Point Numbers)	39
8.1 基本概念与名词解释	39
8.2 指数域: 偏置 (Bias) 的本质与指数	43
8.3 尾数域: 隐含位与有效数字	44
8.4 示例 1: 0.15625 (可精确表示)	48
8.5 示例 2: -7.75 (带符号)	49
8.6 示例 3: 0.1 (无法精确表示)	50
8.7 额外总结	50
8.8 字符类型 (Character Types)	53
8.9 布尔类型 (Boolean)	53
8.10 枚举类型 (Enumerations)	53
8.11 指针类型 (Pointers)	54
8.12 复合与派生类型	54
9 一级标题: 概述	55
9.1 二级标题: 背景	55

2 C 语言的数据基石——变量、常量和字面量

2.1 数据的原形：字面量 (literal) 的概念与初步认识

首先来解释一下什么是字面量 (literal)。在 C 语言中，字面量 (literal) 指的是直接出现在源代码中的“数值”本身，无需标识符（名字）即可使用。说得大白话一点，就是代码中一个又一个所见即所得的数据。如代码 4 所示，代码中出现的一个又一个常量数字如 42、'A'、3.1416、Hi 就是字面量 (literal)。

代码 4: C 语言中字面量的示例

```
1 #include <stdio.h>
2
3 int main(void) {
4     int x = 42;           // 42 是整型字面量
5     char c = 'A';         // 'A' 是字符字面量
6     double pi = 3.1416;   // 3.1416 是浮点字面量
7     printf("%s\n", "Hi"); // "Hi" 是字符串字面量
8     return 0;
9
10 //注意我这里指的不是变量名 x、c、pi，而是等号右边的那些具体数值
11 }
```

在汇编层面，字面量并不占用数据内存空间 RAM（字符串字面量除外），通常情况下数值字面量并不会被放入 RAM(数据存储区)，而是以立即数 (Immediate Value) 的形式直接编码到机器指令中。例如此处指令中的字面量 0x12:

```
1 MOV A, #0x12           ; 汇编语言，将立即数 0x12 加载到累加器 A 中
```

该指令通常在机器码中被编码为:

```
1 74 12           ; 机器码表示: 74 是 MOV A 这一动作的指令代码, 12 是立即数部分
```

这里的立即数 12H 是指令字节的一部分，最终都会随着指令流一起储存于程序存储器 (Flash/ROM) 中，而非 RAM 数据区。

2.1.1 补充：字面量的定义

在 C 语言标准 (如 ISO/IEC 9899:2018, C17) 中,“字面量” (*literal* 或 *literal constant*) 是一个正式存在的术语。它指的是直接出现在源代码中的常量值，无需标识符即可使用。

2.1.2 常见字面量的种类（根据 C17）

字面量类型	示例	说明
整型字面量 (integer literal)	123, 0x7B, 010	十进制、十六进制、八进制整数
浮点型字面量 (floating-point literal)	3.14, 1e-3	双精度浮点数
字符字面量 (character constant)	'A', '\n'	单个字符，本质为 <code>int</code> 类型
字符串字面量 (string literal)	"Hello"	以 <code>\0</code> 结尾的 <code>char</code> 数组

枚举常量 (enumeration constant) 虽然是常量，但它是通过标识符声明的，不属于“字面量”。

2.1.3 与“常量表达式”的区别

- 字面量是 **最简单的常量表达式** (constant expression)，但常量表达式可以包含运算符（如 `2+3`），而 `2+3` 不是字面量。
- 在 C 标准的语法描述中，字面量属于 **primary expression** 类别之一。

2.1.4 与“宏”或“const 对象”的区别

- 宏替换发生在预处理阶段，宏展开后可能变成字面量，但宏名本身不是字面量。
- `const int x = 5;` 中 `x` 是标识符，不是字面量，虽然它可能在编译期有常量值。

这两部分属于“变量与常量”的范畴，后续章节会详细介绍。所以现在看不懂也没事，后续看到相关的章节再回来复习此处内容即可。

2.2 字面量的进制与类型区分

在前面的学习中，我们已经了解了字面量的基本概念。我们清楚了字面量是没有标识符的常量数据，是直接写在代码中的数据本身。但是有一点我们前面没有提及，那就是字面量虽然没有独属于自己的标识符，但是它有自己的类型和进制。那么在详细阐述字面量的类型与进制区分的相关知识之前，我们先回归到我们最熟悉的场景——日常生活中的数字。

在我们日常生活中，我们每天都会接触大量的数据，比如购物、计数。但我们从来不会纠结于数据类型或是进制这些乱七八糟的东西。这是因为在日常生活中，我们普遍使用十进制，同时我们思维也是抽象地、没有物理内存限制的，也就不存在数据类型的问题——我们讨论的就是纯粹的基于数学的数字概念。所以对于我们彼此交流的时候，从来都不会纠结于进制、类型问题。



图 1: 日常生活

但在计算机中可不是这样，首先在计算机底部存储数字使用二进制，而其他八进制、十六进制等进制也是常见的表示方式，再加上由于计算机存储数据的方式不同又诞生了浮点数、整型数、字符。

所以当你冷不丁地给计算机抛出一个数字 42 时，计算机并不能理解这个数字到底是什么进制、类型，如下面这行所示，计算机并不能理解这是十进制的 42，还是八进制的 34，还是十六进制的 68。它更无法理解这到底是 `int`、`double` 还是 `float`。

计算机此时只能按照默认处理规则进行处理，默认情况下这个 42 将被理解为十进制、`int` 类型的数据。但如果你的本意并不是这样，而是想表达其他进制或类型的数据，那么计算机就会误解你的意思。那么在工程场景下，往往就会引发严重的错误。

1

42 //你到底想表达什么呢？

计算机遇到这种情况时，它是无法理解的，因为它不知道这个数字到底是什么进制、类型。这就好像一个麻瓜突然看到一个魔法师念咒语一样（来自哈利波特），它根本无法理解魔法师到底在说什么。对于这种情况，工程师经常形象地戏称为魔法数字（Magic Number）。

所以这里就引出了 C 语言对字面量的具体区分方式。——通过“前缀（`prefix`）”和“后缀（`suffix`）”来对字面量（`literal`）进行类型和进制的区分。

C 语言里面通过“前缀（`prefix`）”和“后缀（`suffix`）”来对字面量（`literal`）进行类型和进制的区分。本节总结了 C 语言中通过前缀（`Prefix`）与后缀（`Suffix`）来区分字面量的进制与类型的方式。内容适用于 C89/C99/C11/C23 标准。

2.2.1 整数：通过前缀区分进制

整数字面量可使用前缀（`prefix`）确定进制，如表 2 所示。

表 1：整数前缀及其含义

前缀	进制	示例
(无前缀)	十进制	123
0	八进制	0123 （十进制 83）
0x / 0X	十六进制	0x123 （十进制 291）
0b / 0B (C23)	二进制	0b1010 （十进制 10）

注意：

- 以 0 开头的整数默认按八进制解析（如 `0123` $\neq$ `123`）。所以在 C 语言中避免使用前导零，以免引起歧义。这算是一个非常经典的错误原因呢。
- C89/C99/C11 没有二进制前缀，直到 C23 才正式加入 `0b` 作为二进制前缀，有些编译器（如 GCC）较早支持。

- C 语言中只有十六进制浮点数可以带前缀，其他进制的浮点数一律不允许带前缀。（具体见后文）。

2.2.2 整数：通过后缀区分类型

整数字面量的类型可由后缀（Suffix）确定。表 3 汇总了常见后缀。

表 2: 整数类型后缀

后缀	类型	示例	中文含义
(无后缀)	int	123	整型（默认类型）
u / U	unsigned int	123U	无符号整型
l / L	long int	123L	长整型
lu / LU	unsigned long	123UL, 123LU	无符号长整型
ll / LL	long long int	123LL	长长整型
组合使用	unsigned long long 等	123ULL, 123LLU	无符号长长整型

如代码 16所示，不同的类型其实意味着存储空间和取值范围的不同。所以当你由于不同的应用场景需要表示不同范围的整数时，可以使用相应的后缀来指定类型。如果这个范围不匹配，编译器会报错或发出警告。

代码 5: 后缀示例

```
1
2 123          // int（默认类型）          -2147483648 ~ 2147483647（32位系统）
3 123U         // unsigned int             0 ~ 4294967295
4
5 123L         // long int                 -2147483648 ~ 2147483647（32位系统）
6 123LU        // unsigned long int        0 ~ 4294967295（32位系统）
7
8 123LL        // long long int            -9223372036854775808 ~ 9223372036854775807
9 123LLU       // unsigned long long int   0 ~ 18446744073709551615
10
11
12 //以上给出的数据范围基于常见的32位和64位系统，
13 //实际范围基于编译器、CPU 架构和操作系统选择的数据模型而异。
```

注意

作为整数字面量后缀：llu 和 ull 完全等价，可以随便使用，没有任何问题。但在 printf 格式字符串中：必须用%llu，不能用%ull。所以我推荐大家统一记忆 llu。

2.2.3 浮点数：无前缀（Prefix 不可用）

浮点数不能使用前缀来表示进制（十六进制除外），不能像整数那样写成八进制、二进制形式：

- 012.3 （非法，浮点不能为八进制）
- 0b1.01 （非法，C 不支持二进制小数）
- 0x3.14 （非法的普通十六进制形式）

因此：浮点字面量不允许使用任何前缀。但凡事总有例外，在 C 语言中，十六进制浮点数就能表示，但其他进制的浮点数一律不允许带前缀。也就是说

C 语言中浮点数字面量只能有两种存在方式，第一种是默认的十进制，第二种是十六进制。

使用前缀表示十六进制浮点字面量（C99 及以后） 从 C99 标准开始，C 语言新增了一类字面量：十六进制浮点字面量（hexadecimal floating constant）。这一类浮点字面量是允许使用 0x 或 0X 前缀的，只是语法与整数常量不同，因此很多教材在入门阶段会直接略去不讲。

(1) 十六进制浮点常量的一般形式

十六进制浮点常量的大致结构可以写成：

<u>0x / 0X</u>	<u>十六进制数字（可带小数点）</u>	<u>p / P + 十进制指数</u>	<u>浮点后缀(f, F, l, L)</u>
十六进制前缀	significand	以 2 为底的指数	类型说明

更口语一点的记忆方式：

0x 十六进制小数部分 p 十进制整数指数 [可选后缀]

其数值等价于：

$\pm (\text{十六进制有效数字}) \times 2^{\text{指数}}$

(2) 若干合法示例

表 3: 十六进制浮点常量示例及其对应值

字面量	对应的十进制值（大致含义）
0x1.2p3	$(1 + 2/16) \times 2^3 = 1.125 \times 8 = 9.0$
0x9A.8p-1	$(154 + 8/16) \times 2^{-1} = 154.5/2 = 77.25$
0x1.921fb6p1f	≈ 3.1415927 (float π)

下面用代码形式再演示一次（需要支持 C99 及以上标准的编译器，例如 `gcc -std=c11`）：

代码 6: 十六进制浮点常量示例

```

1 #include <stdio.h>
2
3 int main(void) {
4     double b = 0x1.8p1;    // 3.0
5     double c = 0x1.fp2;    // 7.75
6     float  d = 0x.ap-3f;   // 0.078125f
7
8     printf("b = %f\n", b);
9     printf("c = %f\n", c);
10    printf("d = %f\n", d);
11    return 0;
12 }
```

(3) 为什么 0x3.14 仍然是“非法写法”？

在上文我们给出例子：0x3.14 “非法的普通十六进制形式”。这个结论在 C99 之后依然成立，原因是：

- 对整数常量来说，形如 0x3.14 带小数点，本来就不是合法的“整数十六进制字面量”；
- 对十六进制浮点常量来说，0x3.14 也不完整，因为缺少必须的 p 或 P 指数部分。

也就是说，0x3.14 只是一个“十六进制有效数字”，不是完整的“十六进制浮点常量”。如果把它补写完整，例如：

0x3.14p0, 0x3.14p+0, 0x3.14p4

这些才是标准允许的、真正合法的十六进制浮点字面量。

(4) 浮点数十进制科学计数法的写法补充

刚才我们前面讨论了十六进制浮点数的写法，顺便补充一下十进制浮点数的科学计数法写法，十进制浮点数除了常见的 3.14 形式外，还可以使用十进制的科学计数法来表示浮点数，例如：

- 1.23e2: 代表 $1.23 \times 10^2 = 123.0$
- 3.14E-2: 代表 $3.14 \times 10^{-2} = 0.0314$

其通用格式为：

尾数部分 e/E 指数部分

其中：

- **尾数部分 (Mantissa)**: 可以是整数或小数（如 1.23 或 3）。
- **e/E**: 作为分隔符，表示“乘以 10 的多少次方”。
- **指数部分 (Exponent)**: 必须是**整数**（如 2、-3），不能是小数。

非法示例：

- e3 （缺少尾数）
- 2.1e （缺少指数）
- 2.1e3.5 （指数必须是整数）

2.3 浮点数的后缀

浮点字面量只能用后缀确定类型，如表 5 所示。

表 4: 浮点数字面量类型与后缀

后缀	类型	示例	字节数（典型）	精度（有效数字）	中文含义
(无后缀)	double	3.14	8 字节	约 15-16 位	双精度浮点数 (默认)
f / F	float	3.14f	4 字节	约 6-7 位	单精度浮点数
l / L	long double	3.14L	10-16 字节	约 18-33 位	扩展精度浮点数

示例：

代码 7: 浮点数后缀示例

```
1 #include <stdio.h>
2
3 int main(void) {
4     3.14f          // float
5     2.718L         // long double
6     1e-3           // double (默认类型)
7 }
```

2.4 前后缀小结

- 整数字面量：前缀区分进制，后缀区分类型。
- 浮点字面量：无前缀，只能用后缀区分类型。
- C99 起提供十六进制浮点字面量（如 0x1.2p3）。

2.5 字面量底层机制揭秘（ARM）

要深入理解字面量在硬件层面的行为，我们需要先了解一些计算机体系结构的基础知识（注：本节内容涉及计算机组成原理，建议读者结合相关背景知识阅读）。

在嵌入式开发领域，ARM 架构占据了统治地位（本书所有示例均基于 ARM 架构）。经典的 ARM 指令集通常采用固定的 **32 位指令长度**。这意味着，每一条机器指令——无论是运算、跳转还是访存——都必须严格限制在 32 个二进制位（4 字节）之内。

这就引出了一个物理上的**根本矛盾**：

- **指令容器的局限性**：一条指令只有 32 位。在这有限的空间内，CPU 必须编码操作码（Opcode）、目标寄存器、源寄存器等控制信息。留给嵌入“字面量数值”的空间往往非常有限（通常不足 12 位）。
- **数据数值的广度与复杂度**：一个标准的 32 位整数覆盖了 32 位全空间；而一个 IEEE 754 标准的浮点数，其编码结构更是复杂且同样占据 32 位或更多。

显然，我们无法将一个任意的 32 位大整数（例如 0x12345678）或者复杂的浮点数（例如 3.14159）直接“塞”进一条同样只有 32 位宽、且还需要包含操作码的指令中。

为了解决这个问题，ARM 架构设计了一套极具智慧的分级处理机制。我们可以将它们形象地比喻为“随身携带”和“去仓库取”两种模式。

2.5.1 小整数：VIP 通道（塞进指令中，随身携带）

(1) **适配情况**：符合 ARM 指令规则（“8 位基数 + 偶数循环右移”）的字面量（例如 0xFF, 0x100）。此时该小整数字面量由于值小简单（可以通过简单的移位操作构造出来），因此可以直接嵌入到指令内部。

(2) **底层机制**：立即数寻址（Immediate Addressing）。

(3) **硬件过程**：

- ① **编码阶段 (Source)**：你在代码中写下 `int a = 12;` 或汇编语句 `MOV R0, #12`。
- ② **编译阶段 (Compile)**：编译器识别出数值“12”符合立即数编码规则，将其拆分为 *Value* (8 位) 和 *Rotate* (4 位)，直接**硬编码**在 32 位机器指令内部。数据此时不再是独立的数据，而是指令的一部分。
- ③ **链接阶段 (Link)**：链接器根据链接脚本（Linker Script），为包含该指令的代码段（.text）分配未来的 **Flash 的特定地址**（提前建立一个映射关系¹）。此时字面量数据并没有独立的地址，它寄生在指令的地址中，或者说它本身就是指令的一部分。

¹这里的映射其实是一个数学术语，简单理解其实就是函数中自变量 x 和应变量 y 的一一对应关系。对应到这里就是地址和指令在编译器的帮助下建立起了一一对应关系，但此时还未进行实际的地址分配，你可以理解为指令（里面包含着字面量数据）和物理地址提前订婚哈哈。

- ④ 烧录阶段 (Flash): 烧录器通过 JTAG/SWD 接口, 按照上一步链接器生成的映射关系, 将这条包含数据的指令二进制流写入单片机的 Flash 存储单元²。
- ⑤ 硬件实体: 此时, 数值在物理上表现为 Flash 中存储该指令的那几个晶体管的状态。CPU 取指时, 数据随指令流直接进入 CPU, 无需额外的内存读取周期。

编译器将数值拆分为 *Value* (8 位) 和 *Rotate* (4 位), 直接硬编码在 32 位指令内部。数据就在指令流中, 随指令流一起存入 ROM/Flash, 无需占用额外的数据内存。

(4) CPU 内部动作:

- ① 取指解码后, 8 位基数被送入 桶形移位器 (Barrel Shifter)。
- ② 移位操作在数据进入 ALU 之前由硬件并发完成。
- ③ 耗时: 0 额外周期 (极快)。

2.5.2 大整数与浮点数: 物流通道 (放到指定地点, 去仓库取)

(1) 适配情况:

- ① 大整数: 无法通过移位规则构造的 32 位整数 (如 0x12345678)。
- ② 浮点数: 几乎所有浮点字面量 (如 3.14)。由于浮点编码过于复杂, 占据的位数过多, 无法像整数那样压缩到指令中。

(2) 底层机制: 文字池³(Literal Pool) + PC 相对寻址。

(3) 硬件过程:

- ① 编码阶段 (Source): 你在代码中写下大数 `int a = 0x12345678;` 或浮点数 `float f = 3.14;`。
- ② 编译阶段 (Compile): 编译器发现这些数据太“笨重”, 无法塞进一条指令, 于是将它们从逻辑中剥离, 统一收集放到代码段末尾的文字池 (Literal Pool) 中。同时, 在原位置生成一条内存加载指令 (整数用 LDR, 浮点数用 VLDR), 利用 PC 相对寻址指向该数据。
- ③ 链接阶段 (Link): 链接器计算出加载指令与文字池中具体数据之间的偏移量 (Offset), 确保 $PC + Offset$ 能精确指向存放该数据的物理地址。

²这个时候你就可以理解为指令 (里面包含着字面量数据) 和物理地址实际结婚了, 哈哈哈哈哈

³注: 在较新的 ARM 架构 (如 Cortex-M 使用的 Thumb-2 指令集) 中, 引入了 MOVW (加载低 16 位) 和 MOVT (加载高 16 位) 指令。这意味着现在的 CPU 也可以通过两条指令直接把一个 32 位大整数“拼”进寄存器, 而不一定非要查文字池。但在经典 ARM 教学和底层原理理解中, “文字池”依然是最核心的概念。

- ④ 烧录阶段 (Flash): 烧录器将“加载指令”和“数据本身”分别写入 Flash 的不同位置。指令在前面的代码区, 数据在后面的文字池区。
- ⑤ 硬件实体: 此时, 数据物理上存在于 Flash 的某个存储单元中。CPU 执行到该行代码时, 必须发起一次对 Flash 的读操作, 将数据从文字池搬运到寄存器中。

(4) CPU 内部动作:

- ① CPU 计算 $PC + \text{偏移量}$ 获得文字池地址。
- ② 发起内存读操作, 将数据搬运到通用寄存器 (R0-R15) 或浮点寄存器 (S0-S31)。

大整数和浮点数字面量这样操作会比小整数耗时, 因为涉及到内存访问, 会比立即数慢 (通常多 1-2 个周期)。

2.6 特殊字面量: 字符串字面量 (String Literals) 底层机制解密

在此前的学习中, 我们建立了“小整数 (VIP 随身携带)”和“大整数 (去仓库取)”的思维模型。字符串字面量 (如 "Hello") 由于其体量巨大, 属于最极端的“仓库存储”模式。

2.6.1 核心矛盾: 因为字符串“太大”, 只能“匿名”

- 指令容器的局限性: 一条 ARM 指令只有 32 位。
- 数据的庞大性: 即便是短字符串 "Hello" 也占据 48 位 (6 字节), 远超寄存器 (32 位) 和指令的承载能力。
- 结论: 字符串绝无可能进入 CPU 指令内部, 它必须完全依赖外部存储空间, 并通过地址进行访问, 和大整数有些类似。

2.6.2 从代码到晶体管: 物理映射流程

这是一个从软件定义到硬件实体的全过程:

- (1) 编码阶段 (Source): 你在代码中写下 `const char* p = "Hello";`。
- (2) 编译阶段 (Compile): 编译器将 "Hello" 的数据 ('H', 'e'...) 归类为 `.rodata` 段。
- (3) 链接阶段 (Link): 链接器查看链接脚本 (.ld 文件), 决定 `.rodata` 段对应芯片的 Flash 地址 (例如 0x0800 1000, 与前文一样, 此时只是提前建立映射关系)。
- (4) 烧录阶段 (Flash): 烧录器通过 JTAG/SWD 接口, 将数据真正写入单片机对应的 Flash 存储单元中。
- (5) 硬件实体: 此时, 字符串物理上存在于 Flash 芯片内部, 如果不进行擦除操作, 掉电后依然存在。

2.7 核心疑问解答

2.7.1 Q1:CPU 读取字符串地址的指令在哪里？和大整数一样吗？

答案：是的，机制非常相似，但多了一层“跳板”。

这正是 ARM 架构处理 32 位常量的标准做法。因为一条指令（32 位）里塞不下 32 位的地址（例如 0x08001000），所以必须使用 **文字池 (Literal Pool)** 技术，而字符串和大整数的区别在于，他们存放在文字池里的内容不同。字符串是地址，而大整数是数值本身。

- **位置：**读取地址的指令（通常是 LDR）确实写在前面的指令流中（代码段.text）。
- **区别：**
 - **大整数 (0x12345678)：**文字池里存放的直接是 **那个整数的值**。
 - **字符串 ("Hello")：**文字池里存放的是字符串在 Flash 中的 **首地址（指针）**。

字符串字面量的执行三步走演示：

- ① **第一步（指令流中）：**CPU 执行 LDR R0, [PC, #offset]。意思是：“去当前指令附近（文字池）帮我取个东西”。
- ② **第二步（文字池中）：**CPU 在文字池里找到了一个 32 位的数据，这个数据是 0x0800 1000（即字符串在 Flash 里的地址）。注意：这里只是拿到了钥匙，还没见到字符串。
- ③ **第三步（访问.rodata）：**把刚刚找到的地址 0x0800 1000 装到 R0 寄存器里。后续指令使用 R0 寄存器里的地址（0x0800 1000）去访问 Flash，最终读出字符 'H', 'e', 'l', 'l', 'o', '\0'，合成一个字符串。

2.7.2 Q2: 编译器把源代码分为几个区域？都映射到 Flash 吗？

答案：主要分为 4 个核心段。并非所有都映射到 Flash。

在嵌入式开发中，内存布局（Memory Layout）通常如下表所示：

表 5: 常见的内存段分布

段名 (Section)	存放内容	Flash (ROM)	RAM (内存)
.text	代码指令、小整数	是 (存储地)	否 (XIP 模式下)
.rodata	字符串字面量、const 常量	是 (存储地)	否 (直接读取)
.data	已初始化的全局变量	是 (存初值)	是 (运行时)
.bss	未初始化的全局变量	否	是 (零初始化)

详细解析：

- **.text 和.rodata:** 这就是纯粹的“不动产”，一直躺在 Flash 里，CPU 直接去 Flash 里取指和读数据。
- **.data (特殊):** 比如 `int count = 10;`。这个 10 必须存在 Flash 里（否则掉电就没了），但程序运行时你需要修改变量，所以启动代码（Startup Code）会负责把这个 10 从 Flash 搬运到 RAM 中。
- **.bss:** 比如 `int buffer[100];`。不需要在 Flash 占空间，启动时直接在 RAM 里清零即可。

2.7.3 Q3: 字符串字面量在内存中具体是如何存储和处理的？

答案：它具有三个核心特性——只读存储、独立地址、类型退化。

① 存储位置：.rodata (只读数据段)

定位：通常存放在 .rodata (Read-Only Data) 段。

硬件映射：在嵌入式系统（如 STM32）中，这一段直接映射在 **Flash (ROM)** 中。

后果：这意味着它是硬件级别的“只读”。如果你试图通过指针修改它（如 `p[0] = 'X';`），会导致硬件错误（Hard Fault）。

② 内存占用：独立线性地址空间

每一个字符串字面量都在内存中拥有唯一的、独立的“门牌号”（线性地址）。哪怕是两个相同的 "Hello"，在某些编译器设置下也可能合并，但它们依然必须占据独立的地址空间，而不能像立即数那样“融化”在指令里。

③ 退化行为 (Decay)

机制：为了解决“搬运成本高”的问题，在表达式中，数组名自动转化为指向首元素的指针。**本质：**字符串字面量 "Hello" 的类型实际上是 `char[6]`（包含结尾的 `\0`）。

退化 (Decay)：在 C 语言中，除了 `sizeof` 和 `&` 操作符外，数组名在表达式中会自动转换为指向其第一个元素的指针。

类型变化：

源码形式： `char[6]` $\xrightarrow{\text{使用时}}$ 实际类型： `char*`

CPU 视角：CPU 处理的不是一串字符，而是一个 32 位的地址值（指针）。

2.7.4 Q4: 如果 CPU 不小心“执行”到了 Literal Pool 里的数据会发生什么？

答案：CPU 会尝试将数据当作指令解码，通常导致“未定义指令异常”。

这是一个非常危险的情况。CPU 是一种 **取指-解码-执行** 的机器，它本身无法区分当前 PC 指向的是代码还是数据。如果程序逻辑出错（例如忘记写 B 跳转指令），导致 PC 滑入 Literal Pool：

- 运气好时：数据对应的二进制位不符合任何有效指令格式，CPU 抛出 **Undefined Instruction Exception**，程序崩溃但能定位错误。
- 运气差时：数据恰好对应一条有效指令（例如 AND 或 MOV），CPU 会错误地执行它，破坏寄存器状态，导致难以调试的逻辑 Bug。

这也解释了为什么汇编器通常把 Literal Pool 放在 **无条件跳转指令** 之后。

2.7.5 Q5: 既然要分散放置，编译器报错“Literal pool out of range”是什么意思？

答案：这意味着两个“文字池”之间的距离太远了，超过了指令的“射程”。

这直接对应了我们在上文中提到的 **PC 相对寻址限制**。

- 假设你写了一个巨大的函数，长度超过了 4KB（在某些 ARM 模式下）。
- 这里的指令试图读取函数末尾 Literal Pool 里的常量。
- 但是指令中的 offset 字段位宽不够，无法编码这么大的距离。

此时必须在代码中间（例如循环或分支的跳转点之后）手动插入一个新的 `.ltorg`，强行在此处建立一个中转站（新的文字池），缩短指令到数据的距离。

2.7.6 Q4: 为什么不直接把所有常量都放在 .data 或 .rodata 段，像普通全局变量那样访问？

答案：为了性能和代码密度。

虽然可以这样做，但并不高效：

- **访问成本**：访问 `.data` 段通常需要先加载数据段的基地址到寄存器，再进行偏移访问，这需要多条指令。
- **文字池优势**：使用 Literal Pool 配合 LDR（PC 相对寻址），通常只需要 **一条指令** 就能把常量加载进寄存器，且不需要占用额外的寄存器来保存基地址。

Literal Pool 是为了解决“如何快速将大立即数加载到寄存器”这一具体微架构问题而生的特殊机制。

2.8 字面量底层硬件原理总结：三种字面量的“物流通道”

我们可以用“物流运输”来形象地理解 CPU 处理这三种字面量的不同方式：

核心逻辑：能随身携带的（小整数）就随身带；带不动的（大整数）就放车厢（文字池）里查表；太占地方的（字符串）就只拿个提货单（地址），去仓库（.rodata）里取。

2.8.1 小整数：VIP 随身通道 (Immediate)

- **机制：**数值直接嵌入在指令代码中（如 `MOV R0, #12`）。
- **特点：**速度最快。CPU 取指时，数据就跟着指令一起进来了，不需要额外的内存访问。
- **限制：**受指令长度限制，只能存很小的数（如 ARM 的 12 位编码）。

2.8.2 大整数：专用物流通道 (Literal Pool)

- **机制：**数值太大塞不进指令，编译器把它存放在代码段末尾的“文字池”里。
- **特点：**查表读取。指令中存放的是“偏移量”，CPU 执行时通过 `LDR R0, [PC, #offset]` 去文字池里把数搬运到寄存器。
- **代价：**多一次内存访问。

2.8.3 字符串：超大集装箱 (Reference/Pointer)

- **机制：**字符串太长，只能存放在独立的 .rodata 段（仓库）。
- **特点：**传递地址。文字池里存的不再是数据本身，而是字符串在仓库里的首地址。
- **流程：**CPU 先去文字池拿地址（钥匙），再用地址去 .rodata 读数据（取货）。这是最慢的，涉及多次访存。

表 6：三种字面量底层机制对比

维度	小整数 (VIP 通道)	大整数 (物流通道)	字符串 (超大集装箱)
典型例子	<code>MOV R0, #0xFF</code>	<code>LDR R0, [PC, #4]</code>	<code>const char* p = "Hi"</code>
数据大小	极小 (< 12 位)	标准 (32 位)	不定长 (通常很大)
存储位置	指令内部 (代码段)	文字池 (代码段末尾)	.rodata (独立数据段)
获取方式	随身携带 (Immediate)	查表读取数值	获取地址 (指针)
C 语言表现	<code>int</code>	<code>int</code>	退化为 <code>char*</code>

2.9 4. 关键思考：指针 vs 数组

基于上述底层机制，理解以下两行代码的本质区别：

1. `char *str = "Hello";`

- 含义：手里拿着一把仓库的钥匙。
- 操作：`str` 变量直接指向 Flash 中的 `.rodata` 区域。

2. `char arr[] = "Hello";`

- 含义：把仓库的货搬运一份到自家客厅。
- 操作：CPU 执行一段搬运指令，将 Flash 中的字符串复制到 RAM (栈) 中。

2.10 字符串字面量：只读段与栈拷贝 (String Literals)

字符串字面量本质是匿名数组，但其存储和生命周期完全取决于引用方式。这是 C 语言中最大的内存陷阱之一。

1. 指针引用 (`const char *p = "ABC";`)

存储位置 (Storage Location)

- 实体位置：字符串常量直接存放在 **Flash (.rodata)** 段。
- 变量本质：`p` 只是一个保存了 Flash 地址的指针变量。

性能与风险 (Performance & Risk)

- 零拷贝：没有数据搬运开销，直接读取 Flash。
- 只读陷阱：试图修改 `p[0]` 会触发硬件错误 (HardFault)，因为 Flash 物理不可写。

2. 数组初始化 (`char arr[] = "ABC";`)

存储位置 (Storage Location)

- 源数据：字符串实体依然存储在 Flash 中作为初始值。
- 运行时副本：在函数执行时，数据会被拷贝到 **RAM (Stack)** 中。

性能特征 (Performance)

- 初始化开销：编译器会生成 `memcpy` 或 `LDR/STR` 指令序列，在栈上复制一份副本。
- RAM 消耗：占用栈空间 (字符串长度 + 1 字节)。
- 可修改性：内容在 RAM 中，可以自由修改。

2.11 C99 复合字面量 (Compound Literals)

- 定义：如 `(struct Point){.x=1, .y=2}`，常用于结构体传参。
- 存储位置：
 - 若在函数内部使用，它是自动变量，存储在 **Stack (RAM)** 中。
 - 每次进入作用域都会重新初始化（即从 Flash 搬运数据到 RAM）。
- 注意：如果不加 `const` 修饰，它们默认是可变的，且占用宝贵的栈空间。

2.12 总结：字面量存储开销对比表

字面量类型	物理存储 (Entity)	RAM 占用	CPU 访问方式
小整数 (Immediate)	指令流 (Flash)	无	随指令流水线直接解吗
大整数/浮点 (Pool)	文字池 (Flash)	无	PC 相对寻址 (LDR)
字符串 (指针)	.rodata (Flash)	仅指针变量 (4B)	慢 (访问 Flash 总线)
字符串 (数组)	Flash → Stack	有 (长度 +1)	快 (访问 RAM)

2.13 3. 例外情况：字符串与大型常量表

字符串字面量、查表数据、数组与较大的常数无法作为立即数编码。编译器或汇编器会为它们分配独立的数据区（通常在 `.rodata` 或代码区的常量段）。例如：

```
char s[] = "HELLO";
```

会生成类似的数据块：

```
48 45 4C 4C 4F 00
```

2.14 4. 立即数大小受体系结构限制

立即数字面量可编码的位数有限。例如在 8051：

- `MOV A, #imm8` 只允许 8 位立即数；
- `MOV DPTR, #imm16` 支持 16 位立即数；
- 更大的常量无法使用立即数，需要放入程序存储器并通过 `MOVC` 等指令访问。

例如：

```
MOV A, #0x1234 ; 不合法
```

必须写为：

```
MOV DPTR, #TABLE
```

```
MOVC A, @A+DPTR
```

2.15 5. 字面量属于编译期常量

字面量均在编译期确定，不需运行时求值。因此它们不会在运行时消耗额外的 RAM 或寄存器资源。

2.16 6. 更严谨的总结合述

数值字面量通常以立即数形式直接编码到指令中，不占用 RAM 数据存储区。这些立即数属于程序存储器的指令字节，而非独立数据块。当字面量无法作为立即数表示（如字符串、数组、查表数据等），编译器将其存储在程序存储器的数据段中，并通过指针或取数指令访问。

但是字面量虽然没有独属于自己的标识符，但是它有自己的类型

单纯的“变量”在嵌入式里是不够用的，我们还需要给它贴标签，告诉编译器如何对待它。我们知道，诸如数字、文字、符号、图形、音频、视频等数据都是以二进制形式存储在内存中的，它们并没有本质上的区别，那么，00010000 该理解为数字 16 呢，还是图像中某个像素的颜色呢，还是要发出某个声音呢？如果没有特别指明，我们并不知道。

2.17 变量与常量的基本定义

2.17.1 变量

变量（Variable）是程序在运行过程中用于存储数据的命名空间。每个变量在内存中都有一个存储单元，用于保存值，其内容可以在程序执行过程中改变。你可以把它暂时理解为一个装满数据的容器或者是一个可以住人的房间。例如：

代码 8: 变量声明示例

```
1 int a = 10;
2 float b = 3.14;
3 char c = 'A';
4
5 //这里声明完变量之后，还用了赋值语句“=”给变量进行了赋值操作
```

2.17.2 常量

其实这个标题存在一个语言误导，虽然在口语或非严格语境下，人们常把 `const` 修饰的变量称为“常量”，但在 C 语言的严格定义中，常量（Constant）和只读变量（Read-only Variable）是有本质区别的

3 C 语言的变量与常量

在 C 语言程序设计中，数据的表现形式主要分为两种：在程序运行过程中其值可以改变的量称为**变量**（Variable），而在运行过程中其值不能被改变的量称为**常量**（Constant）。

理解这两者的底层逻辑，尤其是它们在内存中的存储方式和编译时的行为，对于编写高效、健壮的嵌入式代码至关重要。

3.1 变量 (Variables)

变量本质上是内存中一段已命名的存储空间。每个变量都拥有三个核心属性：

- **名字 (Name)**: 用于在代码中引用该变量的标识符。
- **类型 (Type)**: 决定了该变量占用多少字节的内存，以及数据如何被解释（例如 `int`, `float`, `char`）。
- **值 (Value)**: 存储在内存单元中的具体数据。

3.1.1 声明与定义

在工程开发中，必须严格区分**声明 (Declaration)** 与**定义 (Definition)**：

- **定义**: 为变量分配内存空间。例如 `int count;`。
- **声明**: 仅告知编译器该变量的名称和类型，不分配内存（常配合 `extern` 关键字用于多文件编程）。

代码 9: 变量的定义与赋值

```
1 #include <stdio.h>
2
3 int main(void) {
4     /* 定义变量: 分配 4 字节 (32位系统) 内存 */
5     int sensor_value;
6
7     /* 赋值: 将数据写入对应的内存地址 */
8     sensor_value = 1024;
9
10    /* 初始化: 定义的同时进行赋值 */
11    float voltage = 3.3f;
12
13    printf("Value: %d, Voltage: %.2f\n", sensor_value, voltage);
```

```
14     return 0;
15 }
```

3.2 常量 (Constants)

广义上的“常量”在 C 语言中有多种表现形式。除了直接写在代码中的数值（字面常量），我们还可以通过特定的关键字或预处理指令来定义具有“常量特性”的符号。

3.2.1 字面常量 (Literals)

字面常量是直接书写在源代码中的数值，它们没有内存地址（通常作为立即数嵌入指令中），是真正的“不可修改”。

- **整型常量**：如 100, 0xFF (十六进制)。
- **浮点常量**：如 3.14, 1.23e-4。
- **字符常量**：如 'A', '\n'。
- **字符串常量**：如 "Hello"（注意：字符串常量存储在静态数据区，修改它会导致未定义行为）。

3.3 三种定义“常量”符号的方式

在工程实践中，为了避免代码中出现“魔术数字 (Magic Numbers)”，我们通常会给常量起一个有意义的名字。C 语言提供了三种主要方式。

3.3.1 1. 宏常量 (#define)

预处理指令 `#define` 用于在编译前的预处理阶段进行纯文本替换。

- **优点**：不占用数据内存 (RAM)，没有类型限制。
- **缺点**：无类型检查，调试困难（符号表中不存在），且容易产生副作用。

代码 10: 宏常量的使用与副作用

```
1 #define SQUARE(x) (x * x) // 危险的写法
2 // #define SQUARE(x) ((x) * (x)) // 推荐：使用括号包裹
3
4 int main(void) {
5     int a = 5;
6     /* 预处理展开为: (a++ * a++) */
```

```
7    /* 导致 a 自增两次，结果不可预期 */
8    int result = SQUARE(a++);
9    return 0;
10 }
```

3.3.2 2. 只读变量 (const)

const 关键字修饰的变量称为只读变量 (Read-only Variable)。重要概念：在 C 语言中，const 变量本质上仍然是变量，而非严格意义上的常量。它拥有内存地址和数据类型，只是编译器在语法层面禁止了赋值操作。

- 嵌入式特性：全局 const 变量通常被链接器放置在 Flash (.rodata) 段，节省 RAM 资源。
- 局限性：在 C89 标准中，const 变量不能用于定义数组长度或作为 case 标签。

代码 11: const 只读变量

```
1 const int MAX_RETRY = 3; /* 全局只读，通常存放在 Flash */
2
3 void func(void) {
4     // MAX_RETRY = 5;      /* 编译报错：试图修改只读位置 */
5
6     /* 错误用法：C 语言中 MAX_RETRY 不是常量表达式 */
7     /* int buffer[MAX_RETRY]; */
8 }
```

3.3.3 3. 枚举常量 (enum)

enum 用于定义一组相关的整型常量。它是 C 语言中真正意义上的具名整型常量。

- 优点：编译期确定数值，符号可见（便于调试），逻辑清晰。
- 特点：C 语言的枚举是弱类型的，本质上等同于 int，编译器不强制检查赋值范围。

代码 12: 枚举的使用

```
1 enum State {
2     IDLE = 0,
3     RUNNING, /* 值为 1 */
4     ERROR    /* 值为 2 */
5 }
```

```
5 };
6
7 int main(void) {
8     enum State current_state = RUNNING;
9     // current_state = 100; /* C 语言允许这样做，虽不推荐 */
10    return 0;
11 }
```

3.4 总结与对比

为了在开发中选择合适的方式，下表对比了这三种方式的特性：

表 7：三种常量定义方式对比

特性	#define (宏)	const (只读变量)	enum (枚举)
处理阶段	预处理 (文本替换)	编译/运行期	编译期
内存占用	无 (嵌入指令)	占空间 (Flash/Stack)	无 (直接量)
类型安全	无	有 (强类型检查)	弱 (视为 int)
调试信息	无 (通常不可见)	有	有
主要用途	全局配置、简单的数学常量	接口参数保护、只读查找表	状态机、选项标志

在 C 语言程序设计中，常量（Constant）是指程序在运行过程中，其值不可改变的量。合理地定义常量可以提升代码的可读性和可维护性。

本文对比三种常用常量定义方式：`#define` 宏常量、`const` 常量与 `enum` 枚举常量，并给出实践建议。

(1) 符号常量 `#define`

`#define` 在预处理阶段进行文本替换，指定用一个符号名称来代替一个常量。它不参与类型检查，也不分配存储空间。适合表达编译期已知且简单的固定值。

代码 13: 使用 `#define` 定义宏常量

```
1 #include <stdio.h>
2
3 #define PI 3.14          //经过指定后，本文件中所有PI都会被替换为3.14。
4 #define AREA(r) (PI * (r) * (r))    //定义函数宏，用于计算圆面积。
5
6 int main() {
7     float r = 2.0;
8     printf("Area = %.2f\n", AREA(r));
9     return 0;
10 }    //简单理解：只是个文本替换而已。
```

在程序编译之前，预处理器会将代码中的所有 `PI` 替换为 `3.14`，所有 `AREA(r)` 替换为 `(3.14 * (r) * (r))`。

(2) 常量变量 `const`

`const` 是 C 语言中的一个关键字，表示“只读（只可读，不可改）”。它可以用来修饰变量、指针、函数参数等，使其在程序运行过程中不可被修改。

常量变量具有类型信息，占用内存空间，只是其值不可通过该标识符修改。适合表达接口常量、配置值等。

代码 14: 使用 `const` 定义常量变量

```
1 #include <stdio.h>
2
3 const int    MAX_CONN = 1024;
4 const float PI      = 3.14159f;
5
6 int main(void) {
7     MAX_CONN = 2048; // 编译报错：只读对象不可修改
8     printf("%f\n", PI);
9     return 0;
10 }
```

要注意一件事情: 修饰一个变量, 只是使其在语义上为“只读”。

也就是说编译器虽然会禁止通过该常量名字修改, 但并不保证存储区域物理上不可修改。`const` 变量仍然占内存, 只是不可通过其标识符修改。

(3) `enum` 枚举常量

`enum` 定义一组自定义的整数常量, 往往是一组有着共同特性的数据集合或是状态标签。这里在展开讨论这一常量之前, 先讲解一下枚举的概念。

枚举

枚举 (Enumeration) 是一种用户自定义的数据类型, 它由一组自定义标签的整数常量组成。每个枚举成员都对应一个整数值 (可以为负数), 默认情况下, 从 0 开始递增, 也可以手动指定起始值或各个具体值。枚举的主要目的是为了提高代码的可读性和可维护性。

比如春夏秋冬这四个数据就是一组枚举数据, 它们可以用一个枚举类型来表示; 再比如生活中的通讯录, 就是一个非常典型的枚举数据, 如图 2 所示。

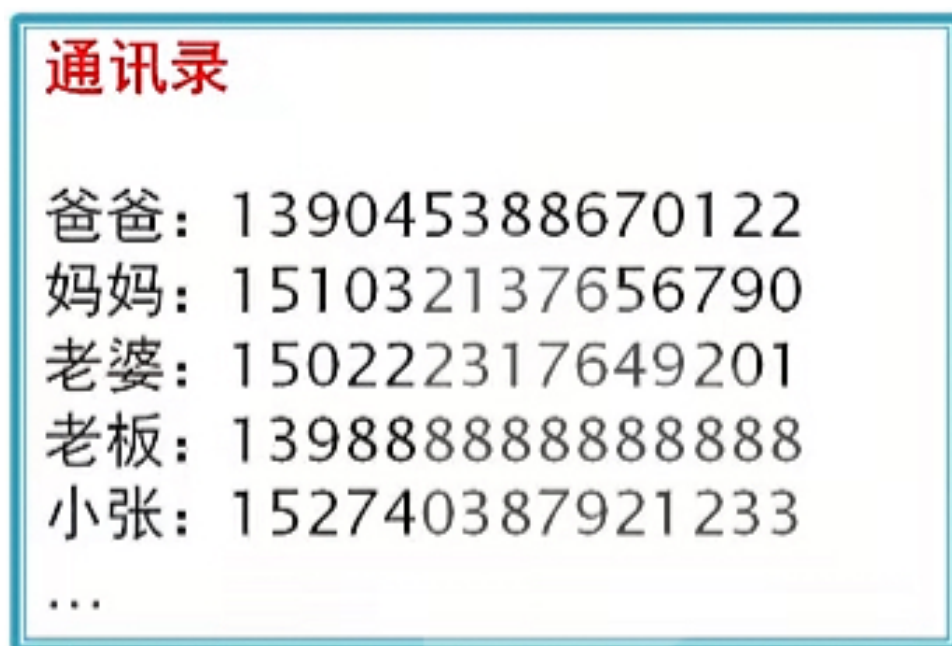


图 2: 生活中的枚举数据: 通讯录

电话号码 11 位数字, 按道理来说, 一共有 10^{11} 种可能性, 但实际上, 我们的通讯录中并不会存入这么多电话号码, 而是只会有其中离散的一部分, 比如你亲人、朋友、同事的号码。此时当我们需要拨打电话时, 只能从通讯录这几个离散的数据里面挑选。

枚举的基本规则如代码 15所示

代码 15: 使用 enum 基本规则

```
1 enum Color { RED, GREEN, BLUE };
2 // 数值若不自定义, 则依次为 0,1,2
3 enum Weekday{ MON=1, TUE, WED, THU, FRI, SAT, SUN };
4 //若自定义了起始值, 则后续依次递增1
5 enum State { ST_INIT=-5, ST_RUN=8, ST_ERR=9, ST_STOP=100 };
6 //也可自定义每个枚举值
7
8 //注意: 枚举数据可以为负数, 但必须为整数。
```

说完了基本规则, 让我们来探讨一下枚举类型变量使用方法, 让我们通过一个示例来加深理解。

代码 16: 使用 enum 定义变量

```
1 #include <stdio.h>
2
3 enum Color { RED, GREEN, BLUE }x; // 定义枚举类型 Color 并声明变量 x
4
5
6 enum Weekday { MON, TUE, WED, THU, FRI, SAT, SUN };
7 enum Weekday this_month; // 声明枚举类型 Weekday 变量 this_month
8 //这与上面Color的声明方式是等价的。
9
10 int main(void) {
11     x = RED; // 给枚举变量 x 赋值 //枚举变量的赋值只能在枚举成员中选择
12     this_month = TUE; // 给枚举变量 this_month 赋值
13     printf("Color: %d, Weekday: %d\n", x, this_month);
14
15     return 0;
16 }
```

在上面的示例中, 我们定义了两个枚举类型, 相信大家已经理解了枚举的基本用法。接下来我们来接着探讨 enum 的常量特性。

enum 常量是编译期符号, 它不同于 const 常量, 也不同于 #define 宏常量在预处理阶段就进行文本替换。enum 常量在编译期被替换为对应的整数值, 不占用任何内存空间, 且只能表示整数类型。枚举变量一旦被声明之后, 其值只能在枚举成员中选择, 不能赋予其他整数值。这一安全特性使得枚举常量非常适合表示离散的状态或类别, 可以有效提高代码的安全性和可读性。

3.5 命名规则与规范

3.5.1 命名规则

对于字符的组成，变量和常量遵循相同的规则。变量名只能由字母、数字和下划线组成，且首字符不能为数字（虽然语法上允许首字符为 `_`，但在实际编程和标准库约定中有一些潜在风险）

- 区分大小写，如 `age` 与 `Age` 是不同变量；
- 不得与关键字（`int`, `return` 等）同名；

3.5.2 代码命名规范

- （1）变量命名规范：变量命名通常使用小写字母和下划线（例如 `total_count`），或者在某些语言中使用驼峰式命名。（例如 `totalCount`）
- （2）常量命名规范：常量通常采用全大写字母，单词之间用下划线分隔。这是为了让开发者在看到常量时，能够立刻识别它们是固定值，而不是可变的数据。例如：`MAX_SIZE`、`PI`、`BUFFER_LIMIT`。

这种约定使得代码更加一致，并且提高了可读性和维护性。

3.6 作用域与生命周期

3.6.1 变量

- （1）局部变量：定义在函数或语句块中，仅在该范围内有效。每次进入函数时该变量都会被重新创建并初始化，生命周期仅限于函数调用期间。当函数或语句块执行结束后，变量随即被销毁，内存空间被回收；
- （2）全局变量：定义在所有函数外部，全文件范围内可访问。其生命周期从程序开始运行到程序结束，在此期间始终占据固定的内存空间。全局变量可被同一文件内的所有函数访问（若需跨文件访问，可使用 `extern` 声明）。
- （3）静态变量（`static`）：使用关键字 `static` 声明。它的作用域视其属于局部变量还是全局变量而定，但其生命周期贯穿整个程序执行过程。

也就是说，即使该变量为在函数或语句块中定义的局部变量，其在函数调用结束后也不会被销毁，而是保留其上一次的值；下次进入函数时会继续使用原来的值，而不是重新初始化。

示例

代码 17: 局部变量与静态局部变量示例

```
1 void foo() {  
2     int x = 0;           // 局部变量  
3     static int y = 0;    // 静态局部变量  
4     x++;  
5     y++;  
6     printf("x=%d, y=%d\n", x, y);  
7 }
```

连续调用 `foo()` 三次的输出为:

```
x=1, y=1  
x=1, y=2  
x=1, y=3
```

由于局部变量在函数调用时被创建, 并在函数结束后自动销毁, 因此每次进入函数时, `x` 都会重新初始化为 1。而静态局部变量 `y` 的生命周期贯穿整个程序运行过程, 因此它的值在每次函数调用后都会保留并继续增加。

3.6.2 常量

- (1) 局部常量: 当常量在函数内部使用 `const` 声明时, 它的作用域仅限于该函数或代码块。这样的常量在函数调用结束后被销毁;
- (2) 全局常量: 当常量在函数内部使用 `const` 声明时, 它的作用域仅限于该函数或代码块。这样的常量在函数调用结束后被销毁。全局变量可被同一文件内的所有函数访问 (若需跨文件访问, 可使用 `extern` 声明)。
- (3) 静态常量 (`static`): 与前面静态变量相似, 如果常量声明为 `static`, 它的生命周期会贯穿整个程序运行期间, 即使它的作用域限于声明的文件或函数。